

```
!pip install xgboost
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

```
from xgboost import XGBClassifier
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
```

```
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
```

```
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

```
df = pd.read_csv(list(uploaded.keys())[0])
```

```
df.head()
```

[Choose Files](#) archive (2).zip

archive (2).zip(application/x-zip-compressed) - 49796 bytes, last modified: 3/17/2026 - 100% done

Saving archive (2).zip to archive (2) (1).zip

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980

5 rows × 33 columns

```
X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1101: RuntimeWarning: invalid value encountered in divide
  updated_mean = (last_sum + new_sum) / updated_sample_count
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1106: RuntimeWarning: invalid value encountered in divide
  T = new_sum / new_sample_count
/usr/local/lib/python3.12/dist-packages/sklearn/utils/extmath.py:1126: RuntimeWarning: invalid value encountered in divide
  new_unnormalized_variance -= correction**2 / new_sample_count
```

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
```

```
rf = RandomForestClassifier()
gb = GradientBoostingClassifier()
xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
```

```
xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
```

```
# Impute NaN values in the scaled data
imputer = SimpleImputer(strategy='mean')
X_train_scaled_imputed = imputer.fit_transform(X_train_scaled)
X_test_scaled_imputed = imputer.transform(X_test_scaled)
```

```
# Encode target variable
le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)
y_test_encoded = le.transform(y_test)
```

```
rf.fit(X_train_scaled_imputed, y_train_encoded)
gb.fit(X_train_scaled_imputed, y_train_encoded)
xgb.fit(X_train_scaled_imputed, y_train_encoded)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any observed value
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/impute/_base.py:635: UserWarning: Skipping features without any observed value
warnings.warn(
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [06:09:08] WARNING: /__w/xgboost/xgboost/src/lea
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
```

```

XGBClassifier
XGBClassifier(use_label_encoder=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None,
              num_parallel_tree=None, ...)

```

```
rf_pred = rf.predict(X_test_scaled_imputed)
gb_pred = gb.predict(X_test_scaled_imputed)
xgb_pred = xgb.predict(X_test_scaled_imputed)
```

```
rf_acc = accuracy_score(y_test_encoded, rf_pred)
gb_acc = accuracy_score(y_test_encoded, gb_pred)
xgb_acc = accuracy_score(y_test_encoded, xgb_pred)

print("Random Forest Accuracy:", rf_acc)
print("Gradient Boosting Accuracy:", gb_acc)
print("XGBoost Accuracy:", xgb_acc)
```

```
Random Forest Accuracy: 0.9649122807017544
Gradient Boosting Accuracy: 0.9473684210526315
XGBoost Accuracy: 0.956140350877193
```

```
rf_auc = roc_auc_score(y_test_encoded, rf.predict_proba(X_test_scaled_imputed)[: ,1])
gb_auc = roc_auc_score(y_test_encoded, gb.predict_proba(X_test_scaled_imputed)[: ,1])
xgb_auc = roc_auc_score(y_test_encoded, xgb.predict_proba(X_test_scaled_imputed)[: ,1])

print("Random Forest ROC-AUC:", rf_auc)
print("Gradient Boosting ROC-AUC:", gb_auc)
print("XGBoost ROC-AUC:", xgb_auc)
```

```
Random Forest ROC-AUC: 0.9955781198820832
Gradient Boosting ROC-AUC: 0.9957418932197838
XGBoost ROC-AUC: 0.9950867998689814
```

```
rf_fpr, rf_tpr, _ = roc_curve(y_test_encoded, rf.predict_proba(X_test_scaled_imputed)[: ,1])
gb_fpr, gb_tpr, _ = roc_curve(y_test_encoded, gb.predict_proba(X_test_scaled_imputed)[: ,1])
xgb_fpr, xgb_tpr, _ = roc_curve(y_test_encoded, xgb.predict_proba(X_test_scaled_imputed)[: ,1])

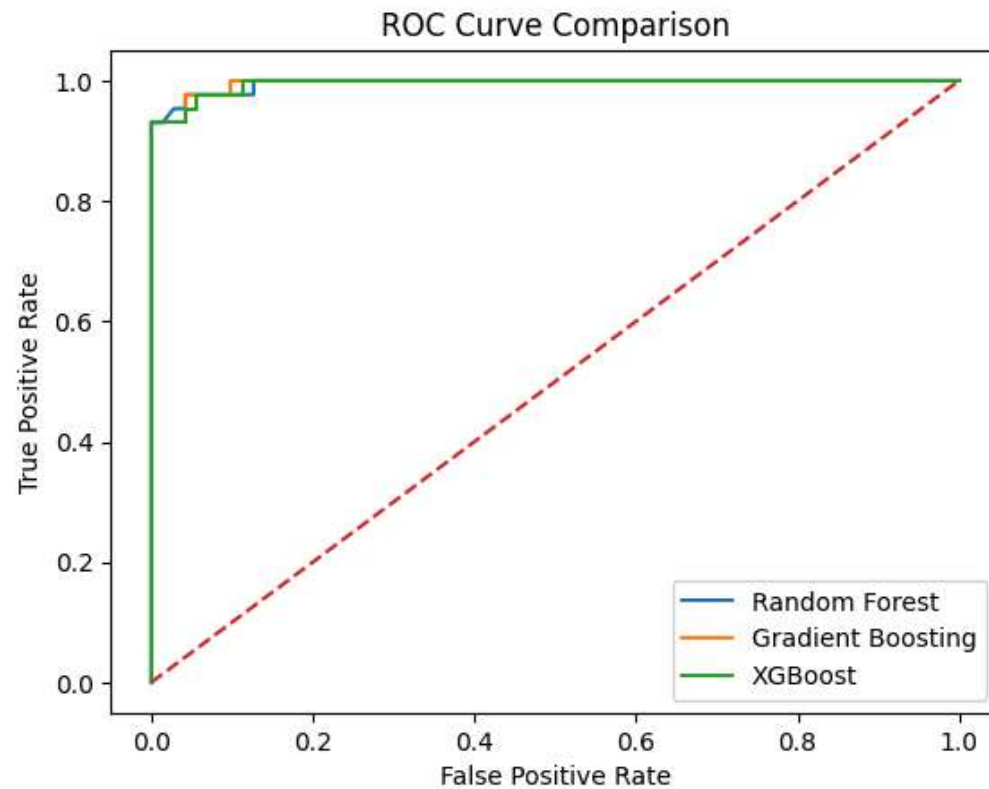
plt.figure()

plt.plot(rf_fpr, rf_tpr, label="Random Forest")
plt.plot(gb_fpr, gb_tpr, label="Gradient Boosting")
plt.plot(xgb_fpr, xgb_tpr, label="XGBoost")

plt.plot([0,1],[0,1], '--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()

plt.show()
```



```
models = ["Random Forest", "Gradient Boosting", "XGBoost"]
accuracy = [rf_acc, gb_acc, xgb_acc]

plt.figure()
plt.bar(models, accuracy)

plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")

plt.show()
```

